

weft 1.1.1

A Modern Literate Programming Tool

User Guide

Cuauhtemoc Pacheco

Derived from nuweb 1.64 by Preston Briggs¹
with contributions by Keith Harwood, Simon Wright,
Marc W. Mengel, John D. Ramsdell, and Gregor Goldbach

March 5, 2026

¹Original nuweb author.

Contents

1	What is weft?	1
1.1	The idea in 30 seconds	1
1.2	Tangle and weave, side by side	1
1.3	Why literate programming	2
1.4	Why weft	2
2	Quick Start	3
2.1	Get weft	3
2.2	Write your first <code>.weft</code>	3
2.3	Tangle: get the code	4
2.4	Weave: get the document	4
2.5	What just happened	4
3	Cheatsheet	5
3.1	Define and reference	5
3.2	Scrap delimiters	5
3.3	Inside a scrap	5
3.4	Fragment names with parameters	6
3.5	Sectioning (scoped names)	6
3.6	Indices, language, and format	6
3.7	Command line: what to produce	6
3.8	Command line: introspection & AI	6
3.9	Per-file flags (on <code>@o</code>)	7
3.10	Escaping, in one line	7
4	The weft Language	8
4.1	Output files and fragments	8
4.1.1	Quoted fragments	8
4.2	Scraps and what lives inside them	8
4.2.1	Inside a scrap	9
4.2.2	Abbreviating long names	9
4.2.3	Tabs and indentation	9
4.2.4	Per-file flags	10
4.3	Fragment names with parameters	10
4.4	Minor commands	10
4.4.1	Indices	11
4.5	Sectioning: scoped names	11
5	Section Markers: Mapping Code to Source	12
5.1	What a marker looks like	12
5.2	How weft knows the comment style	13
5.3	Overriding the language with <code>@l</code>	13

5.4	Defining new languages with @L	13
5.5	The language table	13
5.6	Suppressing markers	14
5.7	A note on highlighting	14
6	Running weft	15
6.1	Invocation and defaults	15
6.2	Choosing what to produce	15
6.3	Weaving to L ^A T _E X	15
6.4	Weaving to Markdown	16
6.5	All command-line flags	16
7	weft for AI and Introspection	17
7.1	The project map: -m	17
7.2	Extract one fragment: -e	17
7.3	Reverse map: -R	17
7.4	Lint the graph: --lint	18
7.5	Preview and translate	18
7.6	A directed workflow	18
8	Debugging Tangled Code	19
8.1	The four-step workflow	19
8.2	Common situations	20
8.3	Doing it automatically	20
9	Worked Examples	21
9.1	One program, several files	21
9.2	Substituting a value inline	21
9.3	One concept across languages	22
10	Restrictions and Acknowledgements	23
10.1	Restrictions	23
10.2	Acknowledgements	23

Chapter 1

What is weft?

weft is a small, language-agnostic tool for *literate programming*: you write one document that interleaves prose and code, and weft turns it into two things — a program a compiler can build, and a document a human can read. It is a modern fork of Preston Briggs’ nuweb, rewritten for clarity and extended with Markdown output, source-mapped section markers, and a toolkit for AI-assisted navigation.

If you have five minutes, read the next two sections and the Quick Start (Chapter 2); that is enough to write your first literate program. The later chapters are a reference you can dip into as needed.

1.1 The idea in 30 seconds

A literate program is a narrative. You explain *what* you are building and *why*, and you drop the code into that explanation in whatever order tells the best story — not the order the compiler demands. Each piece of code is a named *fragment*; you assemble fragments into files by reference.

weft reads that one source file (extension `.weft`) and does two independent jobs:

- tangle** extract the code — write out the real source files a compiler or interpreter consumes. This is the default.
- weave** typeset the document — produce a L^AT_EX (`.tex`) or Markdown (`.md`) file with the prose, the code, and automatic cross-references. Turned on with `-w`.

The same source is both the program and its documentation, so the two can never drift apart.

1.2 Tangle and weave, side by side

One `.weft` file fans out into everything else:

```

                                weft hello.weft  (tangle, the default)
                                +-----> hello.c, util.c, ...
                                (real source, compiled as usual)
hello.weft -----+
(prose + code)  |
                                +-----> hello.tex --> PDF
                                weft -w tex ...   hello.md   (GitHub, ...)
                                (weave)
```

Tangling and weaving are independent: by default weft only tangles, so it fits quietly into a normal build. Ask for a document with `-w`, and a single pass produces both.

1.3 Why literate programming

The premise, due to Knuth [1], is to stop writing code with documentation bolted on and instead write documentation with code woven in — “the difference between performing and exposing a magic trick.” Three things follow:

Order for humans, not compilers. Present a program top-down, bottom-up, or in “stream of consciousness” order — whatever makes it easiest to understand. `tangle` re-assembles it for the machine.

Names instead of detail. A fragment reference like `@<Validate the request@>` replaces a screenful of code with a meaningful name, so structure is visible at every level.

One source of truth. Prose and code live in the same file and are generated together; stale comments become impossible by construction.

1.4 Why weft

`weft` keeps `nuweb`’s best trait — it works with *any* language, or several at once — and modernizes the rest:

Language-agnostic. `weft` never parses your code, so C, Python, Rust, SQL, shell, Protobuf, or a mix all work the same way. What it gives up (automatic prettyprinting, a built-in identifier index) it trades for simplicity and total control over layout.

Two woven formats. $\text{\LaTeX}$ for a typeset book; Markdown (`-w md`) for documentation that renders on GitHub with no $\text{\TeX}$ installation.

Source-mapped output. Tangled files carry *section markers* and `#line` directives, so a compiler error in the generated code maps straight back to the `.weft` source (Chapter 8).

A toolkit for tools. A JSON project map (`-m`), single-fragment extraction (`-e`), reverse source mapping (`-R`), a linter (`--lint`), and compiler-error translation (`--errors`) make `weft` webs navigable by scripts and AI agents alike (Chapter 7).

Small and fast. The command set is tiny, and by default `weft` only tangles, so it stays out of the way of your build.

The rest of this guide is practical: write a first program (Chapter 2), keep the Cheatsheet (Chapter 3) at hand, and reach for the reference chapters when you need the details.

Chapter 2

Quick Start

This chapter walks through a complete literate program from a blank file to a running binary and a typeset document. It uses C for concreteness, but nothing here is C-specific.

2.1 Get weft

If you have the source tree, build the binary once:

```
make          # tangle the .weft sources and compile ./weft
./weft --help # sanity check: prints usage and a directives summary
```

That leaves a `weft` executable in the current directory. Put it on your `PATH` if you like.

2.2 Write your first .weft

Create `hello.weft`. A weft file is mostly ordinary prose (L^AT_EX or Markdown); the only special tokens are the *at-commands*. Type this in verbatim:

```
\documentclass{article}
\usepackage{listings}
\usepackage{color}
\begin{document}
```

We are going to greet the world. The greeting itself is a small reusable fragment, so the main routine reads like a sentence:

```
@d Say hello @{
printf("hello, %s\n", name);
@}
```

The program picks a name and hands it to that greeting:

```
@o hello.c @{
#include <stdio.h>
int main(void) {
    const char *name = "world";
    @<Say hello@>
    return 0;
}
```

```
@}

\end{document}
```

Three commands do all the work: `@d` *defines* a named fragment, `@o` *opens* an output file, and `@<name@>` *references* a fragment so its code is spliced in where the reference sits. The `@{...@}` braces delimit the code (a *scrap*); everything outside them is prose.

2.3 Tangle: get the code

Run `weft` with no flags. That is the default job — tangle:

```
weft hello.weft
```

`weft` writes `hello.c`, assembling the file exactly as the `@o hello.c` scrap describes, with `@<Say hello@>` replaced by its fragment body:

```
#include <stdio.h>
int main(void) {
    const char *name = "world";
    printf("hello, %s\n", name);
    return 0;
}
```

(`weft` also inserts *section markers* — small comments that map each line back to `hello.weft`. They are covered in Chapter 8; ignore them for now.) Compile and run it like any C program:

```
cc -o hello hello.c && ./hello
hello, world
```

2.4 Weave: get the document

Now ask for the document with `-w`:

```
weft -w tex hello.weft      # produces hello.tex
pdflatex hello.tex         # produces hello.pdf
```

The PDF shows your prose with the code set beneath each heading: `<Say hello>` and `<hello.c>` as linked headers, syntax highlighted, with automatic “used in” / “defines” cross-references. To resolve the scrap numbers you run `weft` once more after `LATEX` (`weft` reads them back from the `.aux` file):

```
weft -w tex hello.weft && pdflatex hello.tex
```

Prefer Markdown? Swap the format — no `TEX` needed:

```
weft -w md hello.weft      # produces hello.md, renders on GitHub
```

2.5 What just happened

You wrote *one* file and got two artifacts from it:

```
weft hello.weft           → hello.c (tangle, the default)
weft -w tex hello.weft    → hello.tex → PDF (weave)
weft -w md hello.weft     → hello.md (weave)
```

That is the whole loop. From here: keep the Cheatsheet (Chapter 3) next to you, learn the full command set in Chapter 4, and see bigger programs in Chapter 9.

Chapter 3

Cheatsheet

Everything on one card. Every command begins with the escape character `@` (changeable with `@r`); a file with no `@` is copied through untouched. In these tables a literal at-sign is shown as it is typed in your source.

3.1 Define and reference

<code>@o file flags scrap</code>	Open an output file (tangle target).
<code>@d name scrap</code>	Define a fragment.
<code>@q name scrap</code>	Define a <i>quoted</i> fragment (not expanded on output).
<code>@<name@></code>	Reference a fragment; its body is spliced in.
<code>@i file</code>	Include another <code>.weft</code> file.
<code>@@</code>	A literal <code>@</code> in the output.
<code>@_text@_</code>	Bold the enclosed text (keywords).

A file or fragment may be defined in several pieces; they are emitted in source order. Capital forms `@O` / `@D` / `@Q` are identical but force a page break before the definition when weaving.

3.2 Scrap delimiters

<code>@{ code @}</code>	Code scrap (verbatim; the usual form).
<code>@[text @]</code>	Paragraph-mode scrap (typeset <code>T_EX</code>).
<code>@(math @)</code>	Math-mode scrap (a formula).

3.3 Inside a scrap

<code>@<name@></code>	Expand a fragment inline.
<code>@1 ... @9</code>	Substitute the <i>n</i> th argument.
<code>@f</code>	The current output file name.
<code>@t</code>	The current fragment title (with arguments).
<code>@xlabel@x</code>	Mark a place; refer to it from prose.
<code>@#</code>	At line start: suppress this line's indentation.
<code>@s</code>	Suppress indentation of the next fragment expansion.
<code>@%</code>	Comment out the rest of the line (like <code>L^AT_EX %</code>).
<code>@ ids @}</code>	At scrap end: index these identifiers (<code>@u</code>).

3.4 Fragment names with parameters

<code>@'value@'</code>	A named parameter in a <code>@d</code> name / an argument at a use.
<code>@1...@9</code>	Refer to those parameters inside the body.
<code>name...</code>	Abbreviate a long name (unique prefix).

At each argument position of a reference you may pass: a literal `@'string@'`, a fragment `@<name@>`, an anonymous scrap `@{text@}`, or a forwarded argument `@1...@9`.

3.5 Sectioning (scoped names)

<code>@s / @S</code>	Start / close a local section (isolates names).
<code>@d+ name</code>	Define a fragment visible in <i>all</i> sections.
<code>@<+name@></code>	Reference a global fragment.
<code>@+ / @-</code>	Export / import indexed identifiers across sections.

3.6 Indices, language, and format

<code>@f / @m / @u</code>	Index of files / fragments / identifiers.
<code>@l tag</code>	Set the language for the next <code>@o/@d</code> .
<code>@L name style [+d]</code>	Define a new language (preamble).
<code>@W tex or md</code>	Declare the weave format for this file.
<code>@rx</code>	Change the escape character from <code>@</code> to <i>x</i> .
<code>@v</code>	The version string (defaults to weft's version; <code>-V</code> overrides).
<code>@c</code>	Define / expand a block comment.

3.7 Command line: what to produce

<code>weft f.weft</code>	Tangle only (the default): write the output files.
<code>weft -w f.weft</code>	Tangle + weave, format from <code>@W</code> .
<code>weft -w tex f</code>	Weave <code>L^AT_EX</code> (<code>.tex</code>).
<code>weft -w md f</code>	Weave Markdown (<code>.md</code>).
<code>-t</code>	Suppress weaving.
<code>-o</code>	Suppress tangling (weave only).
<code>-p path</code>	Prefix output file names with <i>path</i> .
<code>-I dir</code>	Add a directory to the <code>@i</code> include search path.
<code>-V str</code>	Set the <code>@v</code> version string.
<code>-n</code>	Number scraps 1,2,3... instead of by page.
<code>-r</code>	Emit hyperlinks (needs <code>hyperref</code> in the preamble).
<code>-v</code>	Verbose progress on <code>stderr</code> .

3.8 Command line: introspection & AI

<code>-m</code>	Emit a JSON map of the whole project.
<code>-m --bodies</code>	... include each scrap's source code.
<code>-m --prose</code>	... include the prose introducing each fragment.
<code>-e name</code>	Extract one fragment (+ its dependencies) as Markdown.
<code>-e name --callers</code>	... show who references it, transitively.
<code>-R file:line</code>	Reverse-map a tangled line to its <code>.weft</code> source.
<code>--lint</code>	Report unused fragments and undefined references.
<code>--diff</code>	Show what tangling would change, without writing.
<code>--errors</code>	Translate <i>file:line</i> compiler errors (from <code>stdin</code>) to source.
<code>--help</code>	Full usage, directives, and AI workflow.

3.9 Per-file flags (on @o)

- d Emit `#line` directives (C/C++/Fortran debugging).
- i Do not indent expanded fragments.
- t Do not expand tabs to spaces.
- cx Comment each fragment in the code (`c = /* */`, `+ = //`, `p = #`).
- s Suppress section markers and `#line` for this file.

3.10 Escaping, in one line

Every literal at-sign in your desired output is doubled: write `@@` to get one . So a Python decorator is `\verb@@proper` a Java annotation `@@Override`, and an email `user@@host.com`.

Chapter 4

The weft Language

The whole of weft is a handful of commands that all begin with the escape character `@`. Everything else in a `.weft` file — $\text{\LaTeX}$ or Markdown prose — is copied through untouched. This chapter is the complete reference: defining files and fragments, the scraps that hold code, parameters, the minor utility commands, and section scoping.

4.1 Output files and fragments

Two commands produce all of weft’s structure. `@o` names an *output file* — a tangle target written to disk. `@d` names a *fragment* — a reusable, named piece of code that is spliced into files (or other fragments) by reference.

<code>@o file flags scrap</code>	Output a file. The name ends at whitespace.
<code>@d name scrap</code>	Define a fragment. The name ends at a return or a scrap.
<code>@q name scrap</code>	Define a <i>quoted</i> fragment (see below).

A file or a fragment may be defined in *several pieces*: repeat the `@o` or `@d` with the same name and weft concatenates the parts, in source order, into one file or one fragment. This is what lets you grow a definition across the narrative — add to a header here, extend a function there — and is shown in the woven output by the `+≡` continuation marker.

The capital forms `@O`, `@D`, and `@Q` behave exactly like their lowercase counterparts but force a page break before the definition when weaving, which is handy for keeping a large scrap together on one page.

4.1.1 Quoted fragments

A *quoted* fragment (`@q`) is defined but *not expanded* when tangling: instead of splicing its body, weft writes it out as a fragment *use*, so the output file can itself be valid `.weft` source. This lets you generate weft files that undergo further processing before their fragments are expanded, while still describing and documenting them in one place. Inside a quoted fragment, `@f` (the file name) is quoted too, so it refers to the finally produced file, not an intermediate one. You may mix `@q` and `@d` definitions of the same name; each part is emitted in order, quoted or expanded as declared.

4.2 Scraps and what lives inside them

A *scrap* is the body of code (or text) attached to a definition. Its delimiters are chosen so you can hold any content exactly, including every blank, tab, and carriage return:

<code>@{ code @}</code>	Verbatim code. The usual form; typeset with <code>listings</code> .
<code>@[text @]</code>	Paragraph mode: a piece of typeset $\text{\TeX}$.
<code>@(math @)</code>	Math mode: a formula, typeset nicely.

Any amount of whitespace (including newlines) may sit between a name and the start of its scrap. A scrap may also appear in the running prose, where it is typeset like a definition but contributes no code to any output file — useful for talking about fragments in the text.

4.2.1 Inside a scrap

Within a scrap body, weft recognises a few sequences:

`@<name@>` Expand the named fragment inline (recursion is an error). When the reference appears *mid-line* — after at least one visible character — section markers, `#line` directives, and name comments are suppressed and the fragment is spliced seamlessly; at the start of a line they are emitted normally (Chapter 5).

`@1 ...@9` Substitute the *n*th argument of the enclosing fragment (empty if not supplied). See §4.3.

`@f` The current output file’s name.

`@t` The current fragment’s title, with parameters replaced by their actual arguments.

`@xlabel@x` Mark a place inside the scrap and bind it to *label* (any text without an `@`). Expands to the scrap’s reference number, so prose elsewhere can point at a spot within a scrap.

`@#` At the start of a line, suppress that line’s indentation — write `@#` before a `#ifdef` so it sits flush left instead of following the surrounding indentation.

`@s` Suppress the indentation of the *next* fragment expansion.

`@%` Comment out the rest of the line, exactly like `%` in ordinary L^AT_EX. Handy for disabling a line of code during testing without disturbing real comments.

`@| ids @}` At the very end of a scrap, list identifiers to record in the `@u` index (§4.4).

4.2.2 Abbreviating long names

A fragment name may be *abbreviated* at a definition or a use: type a unique prefix followed by `...`. So

```
@d Check for terminating at-sequence and return name if found
```

can later be written

```
@d Check for terminating...
```

weft keeps the longest form it sees, and the abbreviation may even appear before the full name. Blanks and tabs inside a name are insignificant — each run collapses to a single space.

4.2.3 Tabs and indentation

When a scrap is written out, tabs are expanded to spaces by default (tab stops every eight columns), and an expanded fragment is indented to match the indentation of its `@<...@>` reference. Take care with layout-sensitive languages (Fortran, Python); the per-file flags below let you turn either behaviour off.

4.2.4 Per-file flags

An `@o` directive may carry flags that tune how *that one* file is tangled — introducing a little language-specific behaviour without affecting the rest of the document:

- `-d` Emit `#line` directives (C/C++/Fortran source debugging).
- `-i` Do not indent expanded fragments (useful for Fortran).
- `-t` Do not expand tabs (useful for `makefiles`).
- `-cx` Comment each fragment in the code: `c=/* */`, `+=//`, `p=#`.
- `-s` Suppress section markers and `#line` for this file.

In most cases the automatic language detection of Chapter 5 is more convenient than these flags, but they remain for explicit control — for example, `-s` keeps bootstrap or generated files free of tangle annotations.

4.3 Fragment names with parameters

Fragment names can carry *parameters*: placeholder values written into the name with the `@'value@'` syntax. Inside the body, `@1 ... @9` refer to them in order of appearance. A reusable validation fragment, for instance:

```
@d Validate field @'name@' type @'string@'
@{if (!data.@1) throw new Error("Missing @1");
if (typeof data.@1 !== '@2')
    throw new TypeError("@1 must be @2");
@}
```

Each use supplies its own arguments:

```
@<Validate field @'email@' type @'string@'@>
@<Validate field @'age@' type @'number@'@>
```

so the first expands to `if (!data.email) ....` Three things to note: the values in the *definition* act as default arguments and as documentation (they make the name readable); a *use* may substitute different values; and if a name is abbreviated with `...`, any missing arguments fall back to the definition's defaults.

At each argument position of a use, four forms are accepted:

- `@'string@'` A literal string (only `@@` is interpreted). Most common.
- `@<name@>` A fragment; its expansion becomes the argument.
- `@{text@}` An anonymous scrap (inherits the caller's arguments).
- `@1 ... @9` Forward an argument from the calling fragment.

4.4 Minor commands

These low-level utilities may appear anywhere in a web:

`@@` Copy a single `@` to the output. (Every literal at-sign in your desired output is doubled in the source.)

`@_text@_` Set the enclosed text bold — for keywords in a scrap.

`@i file` Include another file (nested up to 10 deep). The name is taken literally (no extension appended) and ends at a newline. The current directory is searched, plus any directories given with `-I`, in order.

`@rx` Change the escape character from `@` to `x`. Must precede any scrap definitions.

`@v` The version string — the value set by `-V`, or, by default, weft's own version. Use it to stamp generated files.

`@l tag` Set the language for the next `@o` or `@d` (Chapter 5). Place it on the line before the directive.

@L *name style* [+d] Define a new language mapping. Must appear in the preamble, before any directive that uses it.

@W *format* Declare this file's weave format, `tex` or `md`; used by `-w` when no format is given on the command line. Put it in the preamble.

@c Outside a scrap, define a block comment (all text up to the next command); inside a scrap, expand the last block comment at the current indentation, wrapping lines past 60 columns.

4.4.1 Indices

Three commands generate back-of-document indices, usually each in its own section:

@f Index of output file names.
@m Index of fragment names.
@u Index of user-specified identifiers.

The identifier index is opt-in: you *mark* each identifier at the end of a scrap with `@| FOO BAR @}`. All *uses* of those identifiers inside scraps are then found automatically. Any string without whitespace or `@` works, so even `<<=` can be indexed; an identifier may be marked in more than one scrap. In the index each identifier lists the scraps that use and define it, with defining scraps underlined. (An identifier need not literally appear in the scrap that declares it — it only has to be in that scrap's closing list.)

4.5 Sectioning: scoped names

In a large web the indices and name spaces grow unwieldy, and it becomes easy to reuse a fragment name by accident. *Sections* give each part of the document its own scope: a fragment named in one section is unrelated to a same-named fragment in another, and identifiers do not leak between them. (Scraps in successive sections can still target the same output file.)

@s Start a new local section.
@S Close the current section (following text is the base section).
@d+ *name* Define a *global* fragment, visible in all sections.
@D+ / @q+ / @Q+ The page-break / quoted variants.
@m+ / @u+ Index the global fragments / identifiers.

There are two kinds of section: the *base* section, where everything normally lives, and *local* sections introduced by `@s` and ended by `@S` (or the next `@s`). Base-section fragments are not global — they are reachable only within the base section, but from anywhere in it regardless of intervening local sections. When you really do need to cross a boundary, within a scrap:

@<+*name*@> Expand the *global* fragment of that name rather than a local one.

@+ Like `@|` but *exports* the identifiers to the global realm (not referenced from any single scrap).

@- Like `@|` but *imports* global identifiers into the local realm; uses within the section point here, and the cross-references show where the globals are defined.

The `+` is part of the command, not the name: to define a fragment whose name begins with `+`, leave a space after the command.

Chapter 5

Section Markers: Mapping Code to Source

The code you compile is *tangled output* — files weft assembles from scraps scattered across many concept (`.weft`) files. So a line in `server.js` does not correspond to any file you wrote; it was built from fragments defined in `routes.weft`, `database.weft`, and others. To keep that connection, weft writes *section markers* into the generated code: small comments that name the scrap and point at the exact source file and line it came from.

These markers are the backbone of source mapping. They let the **programmer** browse generated code and jump back to the narrative; they let the **compiler and debugger** report against the `.weft` source (via `#line`); and they let **tools and AI agents** navigate mechanically between output and source (Chapter 7). Because a marker is written as a *comment*, weft must know each file's comment style — and that is the one reason it detects the language.

5.1 What a marker looks like

Each scrap is wrapped in an opening marker (scrap number + source location) and a closing marker (number only), in the language's comment syntax. When fragments nest, each expansion gets its own pair, forming a clear hierarchy:

<code>{... / {:...</code>	Opening / closing delimiter.
Number	Scrap ID, unique across the whole document.
<code>file:line</code>	Source <code>.weft</code> file and line (opening only).

The same scrap, tangled to JavaScript, Python, and C:

```
// {3: routes.weft:42}
app.get("/api/users", handler);
// {3:}

# {5: models.weft:31}
class User: ...
# {5:}

#line 12 "memory.weft"
/* {2: memory.weft:12} */
void *safe_malloc(size_t n) { ... }
/* {2:} */
```

For C-family languages weft also emits a `#line` directive above the marker, so the compiler itself reports errors against the `.weft` file. Scrap numbers are document-global and stable as long as you do not add or remove `@{...@}` blocks before them. Chapter 8 shows how to *use* all this while debugging.

5.2 How weft knows the comment style

To write a marker as a comment, weft needs the file’s comment style. It finds it by a short chain:

file extension $\rightarrow$ language $\rightarrow$ comment style $\rightarrow$ markers in the tangled output

`@o server.js` has extension `.js`; weft looks `js` up in its language table, sees JavaScript uses `//`, and wraps each scrap accordingly. `@o main.c` matches `c` (`/* */` plus `#line`); `@o config.py` matches `py` (`#`). No flags are needed — the extension alone is enough.

5.3 Overriding the language with @l

When the extension is not enough — a `.txt` file that is really SQL — put `@l tag` on the line *before* the directive:

```
@l sql
@o queries.txt
@{SELECT * FROM users;
@}
```

Now the markers in `queries.txt` use `--` comments. `@l` works on `@d` fragments too, and the tag it sets drives both the section markers and the Markdown highlighting. When several sources of language information exist, weft resolves them in order: explicit per-file flags on `@o` win, then an `@l` tag, then the file extension.

5.4 Defining new languages with @L

The built-in table is large but cannot know every language or house style. `@L` registers a new mapping — or overrides an existing one — in the preamble:

```
@L name style [+d]
```

where *name* is the tag recognised in `@l` and file extensions, *style* is the comment syntax (`//`, `#`, `--`, `/*`, or `<!--`), and `+d` optionally enables `#line` directives. For example:

```
@L wgs1 //          % WebGPU shading language: // comments
@L c //            % override the built-in: // instead of /* */
```

User entries are searched *before* the built-in table (up to 32 of them); `@L` is processed in pass 1 and skipped when weaving.

5.5 The language table

weft ships with over 250 entries — languages, extensions, and aliases — grouped by comment style. A tag is matched against `@l` tags and file extensions alike:

Comment style	Tags (representative)
<code>/* */ + #line</code>	c, h, cpp, cc, cxx, hpp, objc, m, mm, cu, cuh
<code>/* */</code>	css, scss, sass, less, cl
<code>//</code>	js, ts, jsx, tsx, java, scala, kotlin, go, rust, rs, zig, swift, dart, php, graphql, proto, sol, glsl, wgs
<code>#</code>	python, py, sh, bash, perl, ruby, r, awk, makefile, cmake, julia, nim, elixir, yaml, toml, dockerfile, terraform, nix, ini
<code>--</code>	lua, sql, psql, haskell, hs, elm, ada, vhd, lean, agda, idris, erlang
<code><!-- --></code>	html, xml, svg, xaml, erb, hbs, twig, md, rst, tex, latex
(none)	json, csv, tsv, txt, diff, patch

If a tag is in neither table, weft falls back to `//` with no `#line`. Tags mapped to the “none” style (`json`, `txt`, ...) produce no markers at all.

5.6 Suppressing markers

Markers are a help, not an obligation. You suppress them when comments would be wrong or unwanted — a data file, a format with no comment syntax, or generated output you want kept pristine:

Per file Put `-s` on the `@o` line (§4.2.4). The language is still detected (so Markdown highlighting works), but no section markers or `#line` directives are written for that file.

Mid-line references A fragment referenced *after* visible text on a line is spliced with no markers at all, so it reads as inline text — `int port = @<port>;` becomes `int port = 4000;`. A reference at the start of a line is a block expansion and gets markers normally. This is what makes fragments usable as inline values; see Chapter 9.

“none” languages Tags like `json` and `txt` have no comment style, so they never get markers.

5.7 A note on highlighting

The language tag has a second, minor use: in the Markdown weave (`-w md`), weft puts it on the code element — `<code class="language-python">` — so GitHub and client-side highlighters (highlight.js, Prism) colour the code. In the L^AT_EX weave, the `listings` package highlights regardless. Highlighting is a by-product of knowing the language; the section markers are the point.

Chapter 6

Running weft

`weft` is a command-line tool. This chapter covers how it is invoked, how to choose between tangling and weaving, the two woven formats, and the full set of flags. (The introspection flags — `-m`, `-e`, `-R`, `--lint`, and friends — have their own chapter, Chapter 7.)

6.1 Invocation and defaults

```
weft flags file...
```

One or more files may be processed at once. A file name with no extension gets `.weft` appended. The input may live in another directory, but the woven document is always written to the current directory. So `weft /foo/bar/quux` reads `/foo/bar/quux.weft` and, by default, writes only the tangled output files.

That default is the important part: **`weft` only tangles unless you ask for a document**. It generates the files named by `@o` and nothing else, so it drops quietly into a normal build. Weaving happens only with `-w`, and then tangling and weaving run in a single pass.

6.2 Choosing what to produce

Three flags select the work:

- `-t` Suppress weaving (override a `-w` in a script).
- `-o` Suppress tangling — weave only.
- `-c` Skip the change-test before overwriting output files.

By default `weft` writes each output file to a temporary location and compares it with the existing file, replacing it only if it changed — so `make` does not rebuild untouched files. `-c` skips that comparison. As an extreme, `weft -to file` scans the input and produces nothing at all.

6.3 Weaving to L^AT_EX

`weft -w tex file.weft` produces `file.tex`. Because scrap numbers depend on where things land on the page, they cannot be known until L^AT_EX has run; `weft` resolves them by reading the `.aux` file on a *second* pass. The usual recipe is therefore:

```
weft -w tex file.weft && pdflatex file.tex && \  
weft -w tex file.weft && pdflatex file.tex
```

`weft` warns when a rerun is needed; if it cannot tell, L^AT_EX warns that labels may have changed. Add `-r` to emit hyperlinks (your preamble must load `hyperref`), and `-n` to number scraps 1, 2, 3... sequentially instead of by page — handy for small webs.

6.4 Weaving to Markdown

`weft -w md file.weft` produces `file.md`, readable on any platform that renders Markdown — GitHub, GitLab, VS Code — with no $\text{\TeX}$ installation. It keeps the essentials (scrap definitions, cross-references, indices) with Markdown-native conventions:

Headers Files as `**"name"** **N** =`, fragments as `**<*Name* N>** =`.

Bodies Code in `<pre><code>` blocks (not fenced), so references inside code can be clickable links.

Cross-references Each scrap has an HTML anchor `<a id="weftN">` and links use `<a href="#weftN">`; footers are blockquotes.

Numbering Sequential (`-n` is forced — Markdown has no pages), no letter suffixes.

Highlighting `<code class="language-X">` when the language is known (Chapter 5).

Fences Cosmetic triple-backtick fences in the source are silently stripped.

Tangling is identical regardless of format; with `-w` a single run produces both the document and the output files.

6.5 All command-line flags

- `-w [fmt]` Weave. `-w tex`, `-w md`, or `-w` alone (uses `@W`).
- `-t` Suppress weaving.
- `-o` Suppress tangling.
- `-c` Do not test output files for change before writing.
- `-p path` Prepend *path* to every output file name.
- `-V str` Value for `@v` (defaults to weft's version).
- `-n` Number scraps sequentially instead of by page.
- `-r` Emit hyperlinks (needs `hyperref`).
- `-x` Include cross-reference numbers in scrap comments.
- `-h opts` Options for the `hyperref` package.
- `-v` Verbose progress on `stderr`.
- `-s` Do not list the scraps that make up each file.
- `-d` Print dangling identifiers (never referenced).
- `--help` Full usage, a directives summary, and the AI workflow.

For the project-map, extraction, reverse-map, lint, diff, and error-translation flags, see Chapter 7.

Chapter 7

weft for AI and Introspection

A literate web is a graph: output files are built from fragments, fragments reference other fragments, and every line of tangled code came from a known place in the source. `weft` exposes that graph through a small set of flags designed for scripts, editors, and AI agents — so a tool can navigate a web precisely instead of reading it top to bottom. None of these commands weave or tangle; they only report.

7.1 The project map: `-m`

`weft -m project.weft` prints a JSON description of the whole web to standard output: every output file and the fragments that compose it, every fragment definition with its source location, every use (cross-reference), and transitive dependencies. Pipe it through `jq` to answer structural questions:

```
weft -m project.weft | jq '.files["server.js"]'      # what builds this file?
weft -m project.weft | jq '.fragments["Validate input"]' # where defined?
```

Two flags enrich the map. `--bodies` adds each scrap's raw source code; `--prose` adds the paragraph of documentation that introduces each fragment. Together they let a tool read a fragment's code *and* its rationale without opening the file:

```
weft -m --bodies --prose project.weft \
  | jq '.fragments["Validate input"].defs[0] | {prose, body}'
```

7.2 Extract one fragment: `-e`

`weft -e "Validate input" project.weft` emits just that fragment and all the fragments it depends on, as a self-contained Markdown document — the exact code a tool needs for one task, with syntax highlighting and source locations, and nothing else. The name matches with the same prefix logic as `@<name...>`.

Add `--callers` to see the other direction — who uses this fragment, transitively, as an indented tree from the `@o` roots down:

```
weft -e "Validate input" --callers project.weft
```

7.3 Reverse map: `-R`

Given a line in a *tangled* file, `-R` tells you where it came from — by reading the section markers already embedded in that file. No `.weft` sources are consulted, so it works in CI and on shipped output:

```
weft -R server.js:47
  {"scrap": 3, "file": "routes.weft", "line": 42}

weft -R server.js      # every scrap region in the file
```

7.4 Lint the graph: `--lint`

`weft --lint project.weft` runs a post-parse check and reports *unused* fragments (defined but never referenced by any `@o` or `@d`), *undefined* references (used in a `@<name@>` but never defined), and, for an undefined reference with a close match, a fuzzy suggestion of the name you probably meant.

7.5 Preview and translate

`weft --diff project.weft` tangles to temporary files and shows a unified diff of what *would* change on disk — a safe preview, and a CI check that the committed output is up to date.

`--errors` closes the loop with your compiler. Pipe a build log in, and every *file:line* it recognises is annotated with the matching `.weft` location (again, read from section markers — no sources needed):

```
make 2>&1 | weft --errors
src/output.c:42: error: undeclared identifier
[weft: iterate/map-output.weft:77, scrap 204]
```

7.6 A directed workflow

These commands compose into a fast loop for navigating a web you did not write — the pattern an AI agent (or a careful human) can follow:

1. **Orient** — `weft --help`, then `weft -m --prose` for the structure and rationale.
2. **Locate** — find the fragment relevant to the task in the map.
3. **Extract** — `weft -e "name"` for exactly that code; `--callers` to see its impact.
4. **Edit** — change the `.weft` source at the reported location (never the tangled output).
5. **Preview** — `weft --diff` to see what tangling changes.
6. **Verify** — re-tangle, build, and pipe errors through `weft --errors` to map any failure back to source.
7. **Check** — `weft --lint` for orphans and typos.

The map and reverse map are complementary: `-m` goes from source to structure, `-R/--errors` go from tangled output back to source. A tool never has to guess.

Chapter 8

Debugging Tangled Code

Chapter 5 explained *what* section markers are and how weft produces them. This chapter is about *using* them — how a marker turns an error in generated code into a precise location in your `.weft` source, for you, for the compiler, and for tools.

8.1 The four-step workflow

Tracing any error back to its source is the same short procedure:

1. **Locate** the failing line in the tangled file — e.g. `server.js:47` from a stack trace.
2. **Read upward** to the nearest opening marker, `// {3: routes.weft:42}`, which names the scrap and its source.
3. **Open** `routes.weft:42`. The scrap is there, wrapped in the prose that explains its intent — which often makes the bug obvious.
4. **Fix, re-tangle, done**. Never edit the tangled file; the markers are there to remind you it is generated.

Nested markers point most precisely. When fragments invoke fragments, markers nest like parentheses. Read the *innermost* enclosing pair — it names the most specific source location:

```
// {3: routes.weft:42}
app.get("/api/users/:id", async (req, res) => {
  // {7: database.weft:18}
  const user = await db.query("SELECT ... WHERE id=$1", [req.params.id]);
  // {7: }
  res.json(user);
});
// {3: }
```

If the bug is inside the `{7:...{7:}` block, go to `database.weft:18`, not `routes.weft:42`.

Let the compiler do it. For C-family languages the `#line` directive weft emits makes the compiler report against the source directly:

```
parser.weft:84: error: 'token_type' undeclared
```

instead of pointing at `parser.c`. Go straight to `parser.weft:84`; the prose around the scrap says what `token_type` is and where it is defined.

8.2 Common situations

Runtime error with a stack trace. Open the file and line it names, read up to the nearest marker, open that `.weft` location.

Wrong output, no error. Search the tangled file for the suspect code; its marker tells you which scrap to examine. Often the prose says one thing (“triple each value”) and the code does another (`* 2`) — bug found.

Which definition produced this line? Fragments defined in several pieces accumulate, but each piece has its own scrap number and source line, so the marker disambiguates them.

Handy habits.

- `grep -n '{[0-9]*: database.weft:' server.js` lists every scrap that came from one source file.
- A closing `{:7}` with no visible `{7:` above means the scrap is taller than your screen — scroll up.
- If your test runner prints code on failure, the markers point straight at the literate source.

8.3 Doing it automatically

You rarely need to trace by hand. `weft` reads the markers for you: `weft -R server.js:47` returns the source location as JSON, and piping a build log through `weft --errors` annotates every *file:line* it recognises with its `.weft` origin. Both work on the tangled output alone — no `.weft` sources needed — which is what makes them ideal in CI and for AI agents. See Chapter 7.

Chapter 9

Worked Examples

Three complete webs, each showing a different strength: splitting one program across files, substituting a value inline, and driving several languages from a single narrative.

9.1 One program, several files

A tiny calculator split into a header and two source files, all from one web. The header fragment is defined once and referenced from both the header file and the implementation, so the declaration can never drift from its use:

```
@d add prototype @{int add(int a, int b);@}

@o calc.h @{
#ifdef CALC_H
#define CALC_H
@<add prototype@>
#endif
@}

@o calc.c @{
#include "calc.h"
@<add prototype@>
{ return a + b; }
@}

@o main.c @{
#include <stdio.h>
#include "calc.h"
int main(void) { printf("%d\n", add(2, 3)); return 0; }
@}
```

`weft calc.weft` writes `calc.h`, `calc.c`, and `main.c`; `@<add prototype@>` expands to the same declaration in both the header (as a prototype) and the implementation (as the function head). Change the signature once and all three files follow.

9.2 Substituting a value inline

A fragment referenced *mid-line* in a scrap is spliced in place, with no surrounding markers or line breaks — so a fragment can act as a single reusable *value*. This is how a version number, a port, or a host name lives in exactly one place:


```

@d port @{4000@}
@d host @{"localhost"@}

@o config.c @{
const char *host = @<host@>;
int port = @<port@>;
@}

```

tangles to a clean, compilable line each:

```

const char *host = "localhost";
int port = 4000;

```

Because the references sit after visible text on the line, `weft` suppresses the usual section markers and `#line` directives and splices the value seamlessly (see Chapter 5). Define the value once; every `@<port@>` picks it up.

9.3 One concept across languages

A web is not limited to one language. A single narrative can define a database schema, the code that uses it, and the script that deploys it, routing each to its own file:

```

@o schema.sql @{
CREATE TABLE users (id SERIAL PRIMARY KEY, email TEXT NOT NULL);
@}

@o app.py @{
import psycopg2
def find_user(conn, uid):
    cur = conn.cursor()
    cur.execute("SELECT email FROM users WHERE id = %s", (uid,))
    return cur.fetchone()
@}

@o deploy.sh @{
#!/bin/sh
psql "$DATABASE_URL" -f schema.sql
@}

```

One `weft` run tangles all three files, each with the right comment style for its language in the section markers, while the prose explains the *one* idea — the users table — that ties them together. The cross-language contract is documented exactly where the languages meet.

Chapter 10

Restrictions and Acknowledgements

10.1 Restrictions

weft is meant to stay simple, so a few restrictions are deliberate. Some may be lifted over time; others seem fundamental.

- **Error handling is not uniform.** In some cases weft warns and continues; in others it halts. This should be regularised.
- **Undefined fragments warn but do not halt.** A reference to a fragment that is never defined is written into the output surrounded by `<>`, which is convenient during development but may change.
- **No @ in file names or index entries.**
- **Fragment names are (almost) any TeX.** You may switch fonts or use math mode in a name, but keep braces, brackets, and dollar signs balanced.
- **Scraps hold anything**, but very long scraps (wide or tall) may not typeset well. A long scrap can be allowed to break across a page if defined with `@0/@D` instead of `@o/@d`; this is a poor default (too many short scraps would break) but a useful per-scrap choice.
- **Output files share the current directory** with the temporary files weft writes for its change-test, so all outputs should be on the same file system — or use `-p` to redirect them.
- **L^AT_EX needs two passes.** Page-based scrap numbers are not known until L^AT_EX has run, so weft must be rerun afterwards for a clean document (much as L^AT_EX itself is rerun for an up-to-date table of contents). weft warns when this is needed; otherwise L^AT_EX warns that labels may have changed. This does not apply to Markdown output, which numbers sequentially and needs no extra passes.

10.2 Acknowledgements

weft is a fork of nuweb, which was written by Preston Briggs and maintained over the years by Marc Mengel, Simon Wright, and Keith Harwood. It in turn drew its name and several features from Norman Ramsey's noweb, and stands on the foundation Donald Knuth laid with WEB.

Many people contributed time, ideas, and debugging to nuweb, among them Osman Buyukisik, Manuel Carriba, Adrian Clarke, Tim Harvey, Michael Lewis, Walter Ravenek, Rob Shillingsburg, Kayvan Sylvan, Dominique de Waleffe, and Scott Warren — and George Greenwade, without whose efforts most of them would never have heard of the tool. Later online contributions came from Walter Brown, Nicky van Foreest, Javier Goizueta, and Alan Karp.

weft carries that work forward with a modernized, concept-organized codebase, Markdown weave output, source-mapped section markers, and the introspection toolkit of Chapter 7 — while keeping the simplicity that made nuweb worth forking.

Bibliography

- [1] Donald E. Knuth, *Literate programming*, The Computer Journal **27** (1984), no. 2, 97–111.